Generative AI for Secure IAM Policy Engineering: A Policy-Constrained Framework for Generating and Verifying Least-Privilege Access Policies

Kanak Patil
Dept. of Computer Science and Engineering (Cybersecurity and Forensics)

Nikhil Mishra
Dept. of Computer Science and Engineering (Cybersecurity and Forensics)

Stuti Singh
Dept. of Computer Science and Engineering (Cybersecurity and Forensics)

Yuval Shah
Dept. of Computer Science and Engineering (Cybersecurity and Forensics)

Course project: Identity and Access Management (IAM), B.Tech. CSE (Cybersecurity and Forensics)

***Abstract*—Identity and Access Management (IAM) policies define which users or services may perform which actions on which resources. Such policies are usually requested in natural language but enforced as precise, machine-readable rules, and errors introduced during this translation can grant excessive privileges, block legitimate work, or violate separation-of-duties requirements. Generative AI, particularly large language models (LLMs), can draft structured policies from human-readable requirements, but it can also produce plausible yet incorrect rules, so generated policies must not be deployed directly. This article proposes a policy-constrained framework for Generative AI-assisted IAM policy engineering. The framework parses a request into structured access requirements, asks for clarification when details are missing, generates a candidate policy in a fixed schema, and passes it through deterministic validation gates for syntax, completeness, least privilege, separation of duties, conflicts with existing rules, and traceability before an authorized human reviewer decides. A deny-by-default principle applies whenever required information is missing or a critical check fails. The design supports role-based (RBAC) and attribute-based (ABAC) representations. An evaluation plan based on policy correctness, constraint-violation rate, completeness, false approval rate, explainability, and generation time is specified. Because the framework is a proposed design, these metrics are intended for future testing and no experimental findings are claimed. The main contribution is a clear separation between probabilistic language generation and security-critical policy enforcement, with human review retained for high-impact decisions.**

***Index Terms*—Identity and Access Management, Generative AI, access-control policy, large language models, least privilege, separation of duties, policy verification.**

I. Introduction

Identity and Access Management (IAM) governs how digital identities are created, authenticated, assigned permissions, monitored, and removed. In an organization, an employee may need to read project documents, a finance officer may need to approve invoices, and an administrator may need to manage accounts. IAM policies express these permissions and help prevent users from reaching information or functions unrelated to their responsibilities.

Two security principles are central to sound policy design. *Least privilege* grants only the permissions needed to complete an assigned task [1]. *Separation of duties* (SoD) splits sensitive responsibilities so that no single person can complete every step of a potentially risky process alone [2]. Both principles are reflected in established access-control guidance, including NIST work on attribute-based access control and access-control verification [3]–[5].

Policy engineering is difficult because business requirements are usually written in everyday language, whereas an access-control engine expects precise rules. Consider the statement: “Finance staff can view invoices, but only a finance manager can approve them, and the same person must not create and approve an invoice.” It contains several permissions and one constraint. If a policy author misses one clause, the resulting rule may grant too much access or block legitimate work. Policies also change as staff change roles, systems are added, and compliance requirements evolve, so small errors can accumulate over time.

Generative AI can help interpret natural-language requirements and draft structured rules. Recent work has examined the use of large language models (LLMs) to extract access-control policy information from natural-language specifications [6]. However, generation alone does not establish that a policy is safe. LLMs are known to produce fluent but unfaithful output [7], so a generated rule must be checked against the intended requirement and the organization’s existing constraints.

The problem addressed in this article is how to use Generative AI to draft IAM policies while preventing incorrect, overly broad, or conflicting rules from being accepted automatically. We propose a framework in which the LLM only generates a candidate policy, while deterministic checks and accountable human approval decide whether it may be deployed.

The contributions of this article are: (1) a staged, auditable workflow and architecture connecting natural-language requirements to structured policy drafts; (2) a set of explicit validation gates for completeness, least privilege, separation of duties, conflicts, and traceability, combined with a deny-by-default rule; (3) a threat model for the generation pipeline; and (4) a reproducible evaluation plan for policy correctness, constraint violations, explainability, and generation time. This

is a design proposal: implementation and empirical evaluation are required before any effectiveness claim can be made.

The remainder of the article is organized as follows. Section II reviews background and related work. Section III states the research gap and problem. Section IV gives the aim, objectives, and research questions. Section V presents the methodology and system architecture. Section VI discusses the threat model and security, privacy, and ethical considerations. Section VII describes the evaluation plan, Section VIII outlines a prototype implementation plan, Section IX presents results, Section X discusses contributions, limitations, and future scope, and Section XI concludes.

II. Background and Related Work

A. IAM Policy Models

Role-Based Access Control (RBAC) associates permissions with roles and then assigns users to those roles. It is useful when job functions map reasonably well to stable permission sets, and its standardized models also support role hierarchies and constraints such as static and dynamic separation of duties [8], [9]. Attribute-Based Access Control (ABAC) makes authorization decisions by evaluating attributes of the subject, resource, requested action, and sometimes the environment. For example, access may depend on a user's department, the sensitivity of a document, the requested operation, and whether the request comes from a managed device [3], [4]. RBAC can simplify administration, while ABAC can express more contextual rules; either model can still be configured incorrectly.

Policy-based systems commonly separate the point where policies are authored and administered from the point where decisions are made and enforced. The XACML reference architecture, for instance, distinguishes a policy administration point (PAP), a policy decision point (PDP), a policy enforcement point (PEP), and a policy information point (PIP) [10]. The framework proposed here adopts this separation: the generative component takes part only in policy authoring and never in run-time access decisions.

B. Policy Verification and Governance

Policy verification checks whether rules contain faults that leak privileges or unintentionally block access. NIST IR 8360 discusses machine-learning-based access-control policy verification and explains that traditional verification approaches can face challenges involving test coverage, complexity, and translation of complex policies into testable models [5]. This establishes that policy validation is a task distinct from policy creation: a policy may be syntactically valid and still violate organizational requirements. Formal approaches complement testing; for example, model checking has been applied to cloud IAM policies to test whether stated security properties hold [11].

Governance requirements add a further layer. Control catalogues such as NIST SP 800-53 include controls addressing least privilege and separation of duties that an organization can map to concrete, checkable policy constraints [12]. NIST's AI Risk Management Framework: Generative AI Profile provides a broader risk-management perspective for the design, development, use, and evaluation of generative AI systems [13]. For IAM policy engineering, this supports treating the model as a component that needs testing, oversight, documentation, and risk controls rather than as an unquestioned authority.

C. Generative AI for Access-Control Policies

Research from NIST has examined using large language models to translate natural-language specifications into access-control policies, which is directly relevant to converting business requirements into machine-readable rules [6]. Research on formal verification of access-control policies also shows the value of checking policies against security properties rather than relying on their apparent correctness [11]. These lines of work suggest that policy generation and policy verification should be treated as connected but distinct tasks.

Two properties of LLMs motivate the architecture proposed here. First, a model may generate content that reads plausibly but is not faithful to its input [7]. Second, when an LLM-based application processes untrusted text, instructions hidden in that text can steer the model's behavior, a risk described as indirect prompt injection [14]. In an IAM setting, a submitted requirement, or a document retrieved to support it, could attempt to widen the generated permissions. Both model input and model output must therefore be treated as untrusted.

The proposed framework combines these ideas into a lifecycle: interpret the requirement, generate a structured draft, run deterministic validation, explain any failures, and require human approval before deployment. Its specific emphasis is the joint checking of least privilege, separation of duties, policy conflicts, and traceability within one review workflow. This is the focus of this project, not a claim that no existing system has addressed these features.

D. Literature Comparison

Table I summarizes the sources that most directly inform this work and indicates how each relates to the proposed framework.

TABLE I
LITERATURE COMPARISON

Source	Focus and useful contribution	Limitation / relevance
Hu, NISTIR 8360 (2021) [5]	Access-control policy verification; treats verification as a way to detect faults that leak or block access.	Does not by itself specify the complete LLM-to-policy workflow proposed here.
NIST SP 800-162 (2019) [3]	Attribute-Based Access Control; defines attributes used in authorization decisions.	Foundational guide, not a Generative AI policy-generation system.
NIST SP 800-205 (2019) [4]	Attributes in access-control systems; implementation considerations.	Needs a separate design for safe AI-assisted policy drafting.
Lawal et al. (2025) [6]	LLMs and natural-language access specifications; translation into policies.	The full method and evaluation should be examined before drawing detailed conclusions.
NIST AI 600-1 (2024) [13]	Risk-management guidance for generative AI.	General AI risk guidance must be mapped to IAM-specific checks.
Gouglidis et al. (2023) [11]	Model checking of cloud IAM policies in a case study.	Needs a model and properties; does not replace requirement clarification or human review.
Greshake et al. (2023) [14]	Indirect prompt injection against LLM-integrated applications.	Motivates treating requirements and retrieved text as untrusted data.
Ji et al. (2023) [7]	Survey of hallucination in natural-language generation.	Motivates validating generated policies rather than trusting fluent output.

III. Research Gap and Problem Statement

Prior work provides foundations for access-control policy verification, attribute-based authorization, and the translation of natural-language specifications into policy drafts. Yet a practical IAM workflow must do more than generate a syntactically valid rule. It must preserve the meaning of the original requirement, detect missing details, enforce least privilege, identify separation-of-duties conflicts, compare new rules with existing policies, and retain a record explaining why a policy was accepted or rejected.

The research gap for this proposed study is the need to evaluate an auditable workflow that connects natural-language requirements, structured policy generation, explicit security constraints, conflict detection, and human approval. This is the focused design gap chosen for this project, not a claim that no prior system has addressed these features.

Problem statement: How can a Generative AI-assisted IAM policy engineering framework convert natural-language access requirements into structured policy drafts while detecting violations of least privilege and separation of duties before deployment?

The study will investigate whether separating language-model generation from deterministic validation can improve the reliability and auditability of policy drafting compared with accepting model-generated policies without equivalent validation.

IV. Aim, Objectives, and Research Questions

Aim: To design and specify a policy-constrained framework that uses Generative AI to assist in creating and validating IAM access-control policies. The objectives are:

O1) Identify common errors and risks that may arise when translating natural-language requirements into IAM policies.
O2) Design a staged workflow for extracting requirements and generating a structured policy draft.
O3) Define validation rules for syntax, completeness, least privilege, separation of duties, and conflicts with existing policies.
O4) Propose measurable tests for policy correctness, constraint-violation rate, explainability, and generation time.
O5) Specify human-approval, logging, and fail-safe controls for policies that fail validation or contain ambiguity.

The study is guided by three research questions:

RQ1: How accurately can a Generative AI system translate natural-language access requirements into structured policy drafts?
RQ2: To what extent do explicit validation gates identify least-privilege and separation-of-duties violations in generated drafts?
RQ3: What trade-offs arise between policy-generation speed, correctness, explainability, and human-review effort?

V. Proposed Methodology and System Architecture

A. Design Principles

Six principles shape the framework. *Generation is separated from enforcement:* the LLM is a drafting assistant, not the final authorization authority; the enforcement engine and established administrative controls remain responsible for actual access decisions. *Deny by default:* if the system cannot interpret a requirement, detects a critical conflict, or cannot verify a constraint, it stops the workflow and requests human review, and it defaults to rejection or non-deployment whenever validation is incomplete. *Deterministic where possible:* security checks are implemented as explicit rules rather than relying on the LLM to judge its own output. *Traceability:* every policy element is linked to the requirement text that justifies it. *Human accountability:* an authorized reviewer approves every policy before release. *Data minimization:* only the information needed for drafting is passed to the model.

B. System Architecture

Fig. 1 shows the proposed architecture, which has seven functional stages. First, an administrator submits a natural-language requirement. Second, a requirement parser extracts the subject, action, resource, conditions, restrictions, and business purpose. Third, the system checks whether required details are missing and asks for clarification instead of guessing. Fourth, an LLM generates a candidate policy in a predefined structured format. Fifth, a deterministic validation engine checks the candidate against syntax rules, approved attributes, least-privilege constraints, separation-of-duties

constraints, and existing policy rules. Sixth, the system produces a report showing the proposed policy, the validation outcome, identified issues, and a trace linking each policy element to its source requirement. Finally, an authorized human reviewer approves, rejects, or edits the draft. Only an approved policy is passed to the deployment process.

The components in Fig. 1 are as follows. *Requirement intake (2)* normalizes the submitted text, limits its length, flags embedded instructions, and marks all content as data rather than as commands. The *requirement parser (3)* extracts a structured representation of subject, action, resource, condition, purpose, and restriction. The *completeness check (4)* compares that representation with the approved attribute catalogue and returns a clarification request when information is missing or ambiguous. The *LLM policy generator (5)* receives only the parsed requirement, the fixed output schema, and the approved attributes; it returns a candidate policy in structured form, and its model version, prompt, and decoding settings are recorded. It holds no deployment credentials and cannot call the enforcement engine.

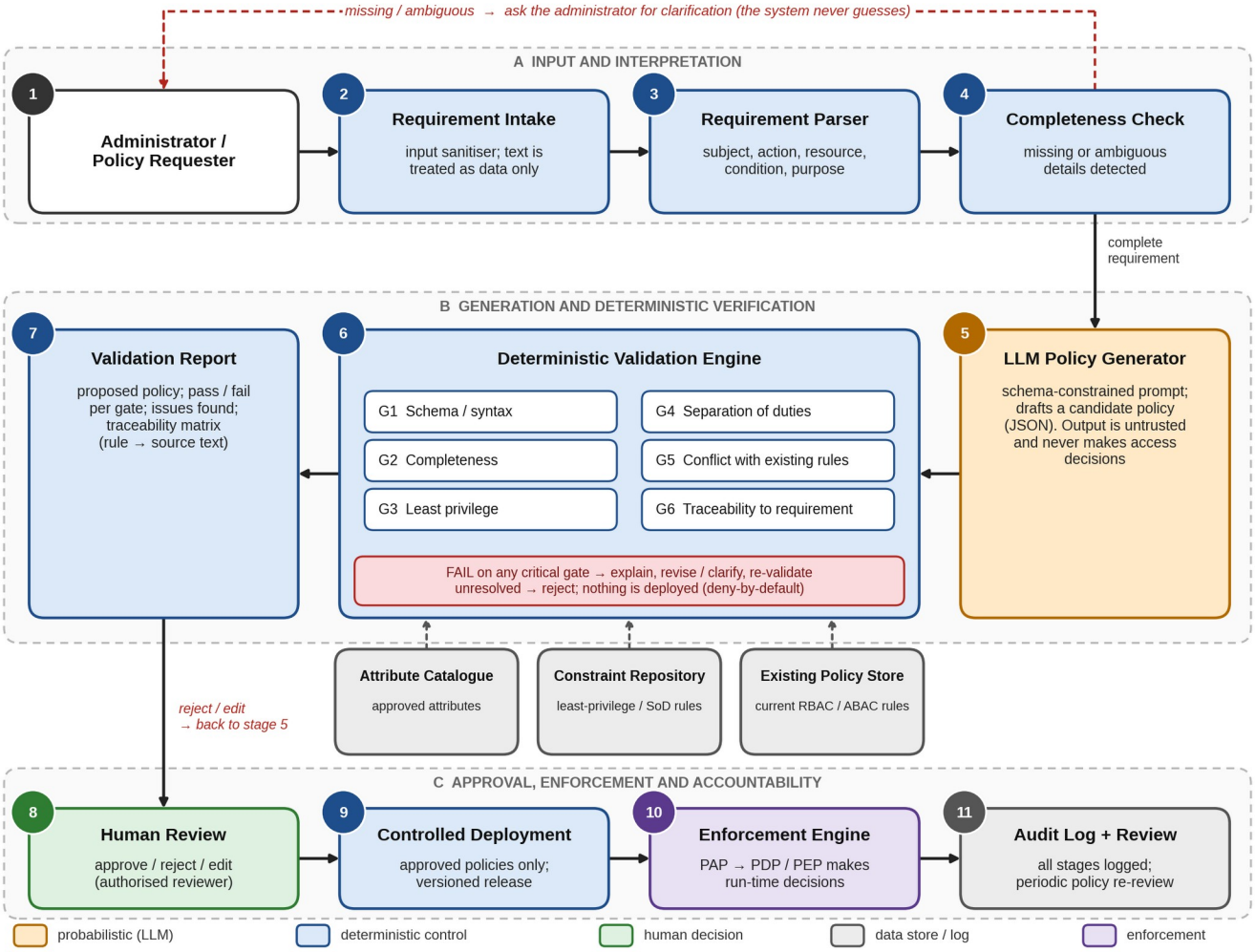


Fig. 1. Proposed architecture and processing flow of the policy-constrained Generative AI framework for IAM policy engineering. Numbers 1–11 give the processing order; colours distinguish probabilistic generation (amber), deterministic controls (blue), human decisions (green), enforcement (purple), and data stores or logs (grey). Dashed red arrows show the failure and clarification paths.

The *deterministic validation engine (6)* runs the gates described in Section V-E against three knowledge stores: the *attribute catalogue* (approved subjects, actions, resources, and attributes), the *constraint repository* (least-privilege and separation-of-duties rules maintained by the organization), and the *existing policy store*. The *validation report (7)* summarizes pass or fail per gate, lists issues, and includes a traceability matrix. The *human review* step (8) lets an authorized reviewer approve, reject, or edit; edits are re-validated. *Controlled deployment (9)* releases only approved, versioned policies to the policy administration point, after which the unchanged *enforcement engine (10)* makes run-time decisions. The *audit log (11)* records every stage and supports periodic review of deployed policies.

C. Workflow and Failure Path

The end-to-end workflow is: natural-language requirement, requirement extraction, completeness check, LLM-generated candidate policy, deterministic validation, human review and approval, controlled deployment, and finally audit logging with periodic review. Fig. 2 expands the verification path. The

candidate policy passes through the validation gates in order, and the human approval gate is reached only when all earlier gates have passed.

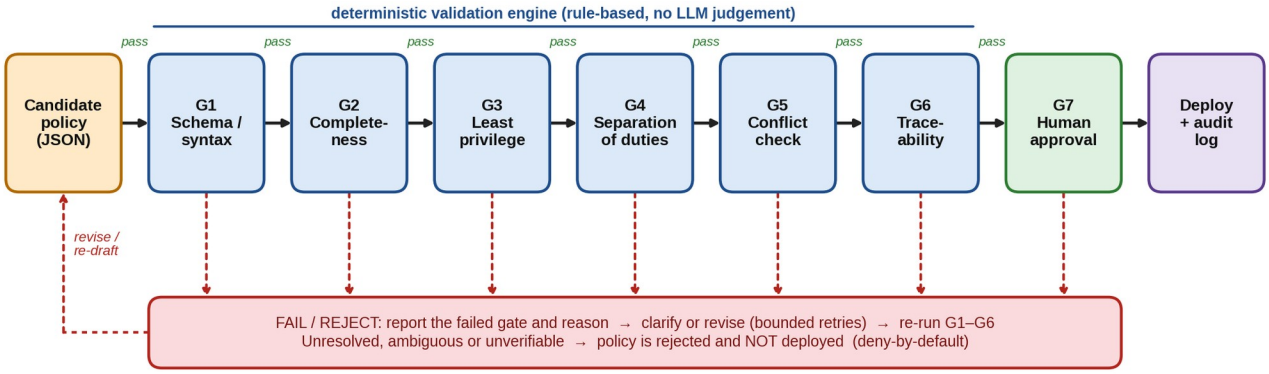


Fig. 2. Validation gate pipeline. A candidate policy must pass gates G1–G6 (deterministic) and G7 (human approval) before deployment. Any failure routes to the reject path, which reports the failed gate, allows bounded revision, and otherwise leaves the policy undeployed.

Failure path: if information is missing or validation fails, the system explains the issue, requests clarification or revises the draft, and then validates again. A maximum number of retries, to be fixed before testing, prevents endless regeneration. If the issue remains unresolved, the draft is rejected and is not deployed. Failures are never silently converted into permissive rules.

D. Policy Representation

For a prototype, the team could use a structured JSON representation with the fields *subject*, *action*, *resource*, *condition*, *effect*, *purpose*, and *source requirement*. The exact schema should be fixed before testing so that output can be checked consistently. RBAC-style rules may use role-to-permission mappings, whereas ABAC-style rules may use attributes such as department, resource classification, device state, or request context [3], [4]. A fixed, enumerated vocabulary for actions and resources reduces ambiguity and lets the validator reason about rules as sets of (subject, action, resource) permissions.

E. Validation Gates

The validation engine should include the gates summarized in Table II. These checks should be implemented as explicit rules where possible rather than relying only on the LLM to judge its own output.

Findings are graded. A *critical* finding (a schema violation, a missing required field, excess privilege, a separation-of-duties violation, or an unresolved conflict) blocks the draft, whereas an *advisory* finding (for example, a redundant rule) is shown to the reviewer but does not by itself block approval. Gates run in the order shown so that inexpensive structural checks precede semantic ones, yet all gates are evaluated even after a failure so that the reviewer receives a complete report rather than only the first problem.

TABLE II
VALIDATION GATES AND FAILURE ACTIONS

Gate	Rule-based check	On failure
G1 Schema / syntax	Output conforms to the fixed schema; attribute names and actions come from the approved vocabulary.	Reject draft; regenerate.
G2 Completeness	Subject, action, resource, and effect are present; no placeholder or wildcard values remain.	Request clarification.
G3 Least privilege	The expanded permission set of the draft is compared with the set entailed by the parsed requirement; extra actions, broader scope, or wildcards are flagged.	Narrow rule; re-validate.
G4 Separation of duties	Incompatible action pairs from the constraint repository (e.g., create and approve) must not be obtainable by one subject on the same resource.	Block; redesign or reviewer exception.
G5 Conflicts	Overlapping permit/deny rules, shadowed or redundant rules, and contradictions with the existing policy store are detected.	Report conflicting rule IDs.
G6 Traceability	Every rule cites a source clause; rules without supporting text are flagged as unsupported.	Remove or justify rule.
G7 Approval	An authorized reviewer's decision, identity, and timestamp are recorded.	No decision, no deployment.

The least-privilege gate (G3) is the most demanding because it requires a notion of "what the requirement entails." In the proposed design, the parsed requirement is expanded into a reference set of permitted (subject, action, resource) tuples, and the candidate is expanded in the same way; any tuple in the candidate but not in the reference set is reported as excess privilege. The separation-of-duties gate (G4) evaluates the constraint repository against role assignments and rule conditions to determine whether a single subject could hold two incompatible permissions on the same resource [2], [12]. The conflict gate (G5) compares each candidate rule with existing policies to find contradictory effects over overlapping scope.

F. Deny-by-Default Decision Procedure

Algorithm 1 summarizes the decision procedure. Every exit path other than explicit human approval ends without deployment.

Algorithm 1. Policy-constrained generation

```
Input: requirement r, catalogue A, constraints K,
       existing policies P, retry limit N
Output: APPROVED(policy) or REJECT / CLARIFY
 1  req <- PARSE(r)
 2  if INCOMPLETE(req, A): return CLARIFY
 3  for attempt = 1 to N:
 4      draft <- LLM_GENERATE(req, schema, A)
 5      F <- VALIDATE(draft, req, A, K, P)
 6      if no critical issue in F: break
 7      req <- REVISE_OR_CLARIFY(req, F)
 8  if critical issue remains: LOG; return REJECT
 9  report <- BUILD_REPORT(draft, F, trace)
10  d <- HUMAN_REVIEW(report)
11  if d = approve: DEPLOY(draft); LOG; APPROVED
12  else: LOG; return REJECT
```

G. Illustrative Example

Consider an organization that wants finance employees to view invoices, finance managers to approve invoices, and no single employee to both create and approve the same invoice. The requirement parser should identify the subjects (finance employee, finance manager), the actions (view, create, approve), the resource (invoice records), and the separation-of-duties condition. The LLM may then draft rules such as those in Listing 1.

Listing 1. Illustrative candidate rules (not a tested result)

```
R1: {"subject": "role:finance_employee",
     "action": "view", "resource": "invoice",
     "effect": "permit", "source": "clause 1"}
R2: {"subject": "role:finance_manager",
     "action": "approve", "resource": "invoice",
     "condition": "creator != approver",
     "effect": "permit", "source": "clauses 2-3"}
```

The validation engine then checks whether the generated rules accidentally allow all finance employees to approve invoices, allow an employee to create and approve the same invoice, or omit a required restriction. For instance, a flawed draft containing a rule that permits *finance_employee* to *approve* invoices would be flagged by G3 as exceeding the requirement and by G6 as having no supporting clause; a draft of R2 lacking the creator-not-approver condition would be flagged by G4. If a rule cannot be represented unambiguously, the system asks the administrator to clarify it. This is an illustrative scenario, not a tested result.

VI. Threat Model and Security Considerations

A. Assets, Adversaries, and Assumptions

The protected assets are the deployed policy set, the constraint repository, the attribute catalogue, the audit log, and the confidentiality of submitted requirements. Considered adversaries include a malicious or careless requester who submits crafted text, content retrieved from untrusted documents, a compromised or careless reviewer, and an external model provider that should not receive sensitive data. The design assumes that the validation engine, constraint repository, and enforcement engine are access-controlled and integrity-protected, and that reviewers are authenticated and authorized. It does not defend against a fully compromised enforcement engine.

B. Threats and Mitigations

Table III maps the main threats to the framework's mitigations.

TABLE III
Threats and Mitigations

Threat	Mitigation in the framework
Prompt injection through requirement text or retrieved documents [14]	Intake treats all text as data; schema-constrained output; validation is independent of the model; the LLM has no credentials or tool access.
Over-permissive generation (wildcards, broad scope)	Least-privilege gate G3; wildcards rejected by G2; deny-by-default.
Missed separation-of-duties constraint	Gate G4 over a maintained constraint repository; reviewer checklist.
Hallucinated rules or attributes [7]	Approved vocabulary (G1); traceability to source clauses (G6).
Conflict with existing policy	Gate G5 against the existing policy store.
Leakage of sensitive data to an external model	Data minimization and redaction; approved model endpoints only; no secrets or credentials in prompts.
Reviewer rubber-stamping	Report highlights failed or ambiguous items; second reviewer for high-impact changes; decisions logged.
Tampering with logs or constraints	Access control, versioning, and append-only audit logging.
Model drift or non-determinism	Record model version, prompt, and settings; re-run the benchmark after any change.

C. Privacy, Accountability, and Ethics

Least privilege and *separation of duties* are enforced as explicit checks rather than as prompt suggestions. *Human accountability:* high-impact policy changes must require approval by an authorized reviewer. *Confidentiality:* secrets, personal information, credentials, and internal policy documents should not be sent to an unapproved external model. *Logging and auditability:* the system records the input requirement, model version, generated draft, validation outcomes, reviewer decision, and deployment identifier while minimizing sensitive data. *Bias and explainability:* the evaluation should check whether equivalent requests are handled consistently, and the system should explain why a policy passed, failed, or requires clarification. *Fail-safe operation:* ambiguous or unverified policies are never deployed automatically. *Authorized testing:* only synthetic or approved data should be used, and only in environments for which the team has permission. These safeguards align with the general risk-management approach in NIST's Generative AI Profile [13] and with established access-control verification principles [3]–[5].

VII. Evaluation Plan

A. Benchmark and Conditions

The project should evaluate the proposed workflow using a small, documented benchmark of synthetic or legally obtained access requirements. The benchmark should include ordinary

permissions, ambiguous requests, overly broad requests, missing fields, conflicting rules, and separation-of-duties cases. No real employee identities or confidential organizational policies should be used without authorization. Each test case should have an expected policy or expected validation outcome defined before testing.

At least two conditions should be compared, as shown in Fig. 3: (A) candidate policies generated by the model without the proposed validation pipeline; and (B) candidate policies processed through the validation pipeline. The same input cases are used in both conditions, and a reviewer independently assesses whether the outputs satisfy the requirements. To make the process reproducible, the team should record the model and version, prompt, test case, generated output, validation findings, reviewer decision, and execution time.

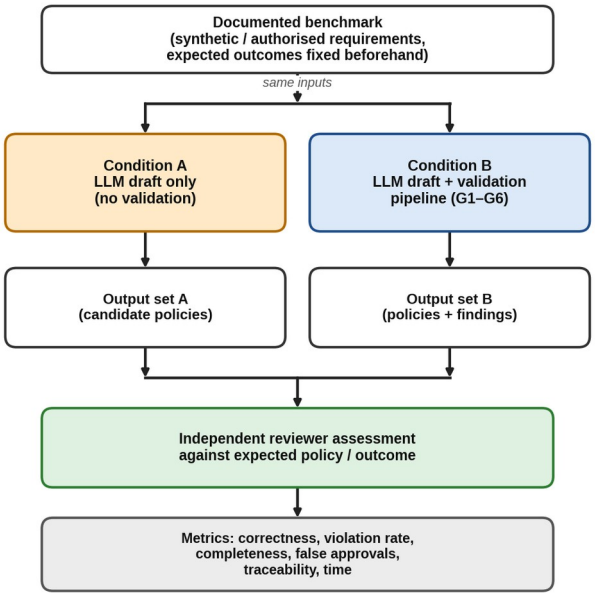


Fig. 3. Proposed evaluation design comparing unvalidated generation (A) with the validation pipeline (B).

B. Metrics

Table IV defines the proposed metrics. The false approval rate is a key safety measure and should be minimized.

Formally, let N be the number of benchmark cases. For each condition, let N_c be the number of policies whose required permissions and restrictions are all correctly represented, N_v the number that violate least privilege or separation of duties, N_u the number of unsafe or incorrect drafts, and N_f those unsafe drafts that nevertheless pass every gate. Then correctness is N_c/N, the constraint-violation rate is N_v/N, and the false approval rate is N_f/N_u. Completeness is the ratio of correctly captured fields and constraints to required ones. Because LLM latency can be skewed, generation time should be reported as a median with its range rather than only as a mean.

TABLE IV
PROPOSED EVALUATION METRICS

Metric	How to measure it	Why it matters
Policy correctness	Share of cases in which all required permissions and restrictions are correctly represented.	Shows whether the policy matches the request.
Constraint-violation rate	Generated policies violating least privilege or SoD divided by policies evaluated.	Measures security-rule failures.
Completeness	Share of required policy fields and constraints correctly captured.	Finds omissions.
False approval rate	Share of unsafe or incorrect drafts that pass all validation gates.	Key safety measure.
Explainability / traceability	Reviewer checklist: is each policy element linked to the requirement and a validation reason?	Supports audit and debugging.
Policy-generation time	Elapsed time from submitted requirement to review-ready draft; human-review time reported separately.	Shows efficiency without hiding review effort.

C. Analysis Plan and Validity

Results should be reported as counts and proportions per condition and per case category (ordinary, ambiguous, over-broad, conflicting, SoD), with the validation gates summarized in a confusion matrix of correct and incorrect drafts versus accepted and rejected outcomes. If more than one reviewer is used, inter-reviewer agreement should be reported. Threats to validity include a benchmark that is too small or unrepresentative, reviewer bias, differences between model versions, and rules that the validator cannot express. These limitations should be stated alongside any numerical findings.

Table V links each research question to the framework components that address it and to the metrics that would answer it. This mapping keeps the evaluation focused on the stated questions rather than on convenient measurements.

TABLE V
RESEARCH QUESTIONS, COMPONENTS, AND METRICS

RQ	Framework components examined	Metrics used to answer
RQ1	Requirement parser, completeness check, LLM generator, gates G1, G2, and G6.	Policy correctness; completeness; traceability.
RQ2	Gates G3 (least privilege), G4 (separation of duties), and G5 (conflicts).	Constraint-violation rate; false approval rate.
RQ3	Whole pipeline, including retries and human review.	Policy-generation time; reviewer effort; explainability / traceability.

Important: no numerical results are claimed in this article. Values may be inserted only after the tests have actually been run; if no prototype is implemented, this section remains an evaluation plan that identifies the experiments needed to validate the approach.

VIII. PROTOTYPE IMPLEMENTATION PLAN

If a prototype is built, a lightweight service could implement the pipeline in a general-purpose language. The intake and

parser modules would produce a structured requirement object; the generator module would call an approved LLM endpoint with a fixed prompt template, a fixed output schema, and recorded decoding settings; and the validation module would implement each gate as an independent function that returns a pass or fail result with a reason and the identifiers of the affected rules. Schema validation would use a standard JSON-schema library, and the permission-set comparisons for G3 and G4 would operate on expanded (subject, action, resource) tuples.

Integration with existing infrastructure should be incremental. In an initial deployment the framework would run in an advisory mode, producing validated drafts and reports for administrators without any automatic release, so that reviewers can compare its findings with their own judgment. The structured output can later be exported to the native policy language of the target enforcement engine, provided the export step is itself covered by tests and by the same approval gate.

The prototype would be tested only in a sandboxed environment with synthetic data and a test enforcement engine. Each run would write a structured log entry containing the fields listed in Section VII. For high-risk policy classes, an additional formal check such as model checking could be added in later work [11]. This plan describes intended work and does not describe an existing implementation.

IX. Results and Analysis

This section reports what happened when the prototype of Section VIII was run in a sandbox with synthetic data and the finance example of Section V-G. The draft generator was a deterministic simulator that can inject six typical model errors: an over-broad rule, a wildcard, a missing separation-of-duties condition, an invented resource, a conflicting rule and an omitted rule. No real LLM was used. The figures therefore show how the validation gates behave on known faults; they are not an estimate of how often a real model makes these errors.

A. Demonstration Scenarios

Fig. 4 shows the first-attempt gate outcomes of six scenarios. The faithful draft (S1) passed G1–G6 and was deployed after reviewer approval. In S2 the simulated model also granted finance employees the approve action; G3, G4 and G6 failed, the findings were returned to the generator, and the second draft passed. In S3 and S4 the fault persisted through all three attempts, so the draft was rejected before it reached a reviewer. S3 failed only G4: the manager role inherits the employee's create right, so an unguarded approve rule breaks separation of duties. S4 used a resource outside the approved vocabulary and failed G1, G3 and G6. The vague request in S5 produced clarification questions instead of a draft. In S6 a sentence ordering the system to grant admin rights was flagged at intake and dropped, and the rest of the requirement gave the same policy as S1. The audit log of the six runs (25 entries) passed its hash-chain check, and queries against the deployed policy returned the expected permit or deny decisions, including a deny when creator and approver were the same person.

	G1	G2	G3	G4	G5	G6	G7	Final
S1 Faithful draft	P	P	P	P	P	P	A	Approved
S2 Over-broad	P	P	F	F	P	F	A	Approved (2 tries)
S3 Missing SoD	P	P	P	F	P	P	–	Rejected (3 tries)
S4 Hallucinated	F	P	F	P	P	F	–	Rejected (3 tries)
S5 Vague requirement	–	–	–	–	–	–	–	Clarify
S6 Injected text	P	P	P	P	P	P	A	Approved

P pass, F critical fail (first attempt), A reviewer approved, – not evaluated or not reached

Fig. 4. First-attempt gate outcomes for the six demonstration scenarios (S1–S6).

B. Fault-Injection Comparison

Each fault type was applied to the same seven permission cases, together with a fault-free control run. Fig. 5 counts the unsafe drafts that were accepted. Without validation (Condition A) every unsafe draft is accepted, which is 37 drafts: 7 for each of five fault types and 2 for the missing condition, because only two of the seven cases contain a separation-of-duties clause. With gates G1–G6 (Condition B), none of the 30 drafts with over-broad, wildcard, missing-condition, invented or conflicting rules was accepted. The exception is the omitted rule: 4 of 7 drafts passed, because a policy that grants too little is only an advisory finding in G6, while the other 3 were single-rule drafts that became empty and were stopped by G1. The false approval rate N_f/N_u is therefore 4/37 (10.8%), all of it from under-provisioned drafts. Treating an uncovered requirement clause as a critical finding could close this gap; its effect on reviewer effort was not measured here.

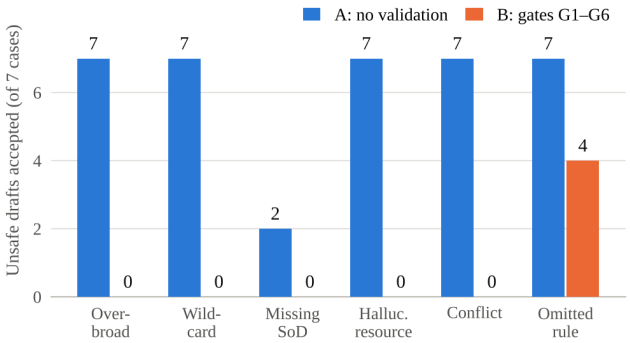


Fig. 5. Unsafe drafts accepted without validation (A) and after gates G1–G6 (B), by injected fault type (7 cases each).

C. Gate Attribution and Cost

Fig. 6 shows how many of the seven drafts raised a critical finding at each gate. Most faults were caught by several gates at once; over-broad rules, for example, were flagged by G3

and G6 in all seven drafts and by G4 in six, which gives some redundancy. A missing separation-of-duties condition is the opposite case: only G4 detects it, so that gate carries the whole protection against this error. One fault-free draft was also blocked (1 of 7 correct drafts). It was the manager approval case whose requirement named only an edit and approve conflict; G4 blocked it because the constraint repository also forbids create plus approve and the manager inherits the employee's create right. This is a block by an organizational rule that the requirement did not mention and would go to the reviewer-exception path, but it counts as a false rejection against the case's expected output. Validation itself was cheap, with a median of 0.20 ms and a maximum of 0.35 ms per draft over 49 runs, so generation and human review would dominate end-to-end time.

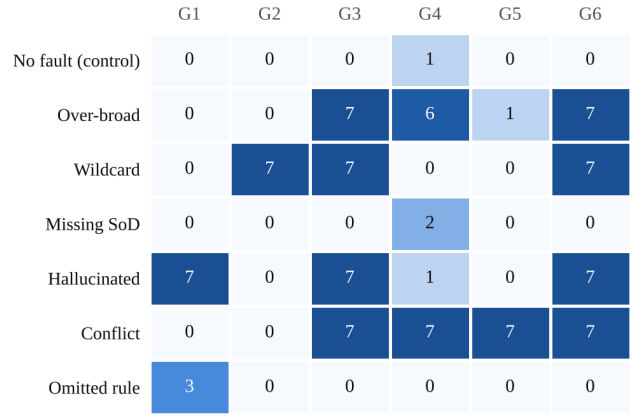


Fig. 6. Drafts (out of 7) with a critical finding at each gate, by injected fault type.

D. Limits of These Results

These results come from seven synthetic cases and scripted faults, so they show that the gates behave as designed, not how effective the framework is with a real LLM. They address RQ2 only for the injected faults. RQ1 (translation accuracy) and RQ3 (speed, correctness and review effort) still need a real model, a larger benchmark and timed reviewers, as set out in Section VII.

X. Expected Contributions, Limitations, and Future Scope

Expected contributions are a documented architecture, a structured policy representation, a set of deterministic validation gates, an illustrative IAM scenario, a threat model, and a reproducible evaluation plan. If a prototype is built, it may also provide measured evidence about which error types the validation stage detects. Until testing is completed, these are expected contributions rather than proven outcomes.

The framework has limitations. Natural-language requirements may be incomplete or contradictory. A validator can only check constraints that have been formally encoded. A language model may generate different drafts for the same request, and validation rules may not capture every organization-specific policy. Human reviewers can also make mistakes. The framework therefore reduces reliance on unverified generation but cannot guarantee that every policy is secure.

Future work could test the design against larger and more diverse benchmarks, support multiple policy languages, integrate with a test IAM environment, measure inter-reviewer agreement, assess model changes over time, and explore formal verification for high-risk policy classes.

XI. Conclusion

This article proposes a policy-constrained approach to Generative AI-assisted IAM policy engineering. The problem is not simply producing a policy draft; it is ensuring that the draft faithfully represents the business requirement and respects security constraints. The proposed architecture separates language-model generation from deterministic validation and human approval, and its checks cover completeness, least privilege, separation of duties, policy conflicts, and traceability. A benchmark-based evaluation plan is defined using policy correctness, constraint-violation rate, false approval rate, explainability, and generation time. The framework has not been experimentally validated in this draft, so its effectiveness remains a question for implementation and testing. The central design principle is that Generative AI may accelerate policy drafting, but security-critical authorization rules must remain subject to explicit controls, accountable review, and evidence-based evaluation.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to the management of Dr. Vishwanath Karad MIT World Peace University, Pune for their encouragement and support throughout this research work.

REFERENCES

[1] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proc. IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975.

[2] D. D. Clark and D. R. Wilson, "A comparison of commercial and military computer security policies," in *Proc. IEEE Symp. Security and Privacy*, Oakland, CA, USA, 1987, pp. 184–194.

[3] V. C. Hu, D. Ferraiolo, D. R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," NIST Special Publication 800-162, updated Aug. 2019. doi: 10.6028/NIST.SP.800-162.

[4] V. C. Hu, D. Ferraiolo, and R. Kuhn, "Attribute Considerations for Access Control Systems," NIST Special Publication 800-205, Jun. 2019. doi: 10.6028/NIST.SP.800-205.

[5] V. C. Hu, "Machine Learning for Access Control Policy Verification," NIST Interagency/Internal Report 8360, National Institute of Standards and Technology, Sep. 2021. doi: 10.6028/NIST.IR.8360.

[6] S. Lawal, X. Zhao, A. Rios, R. Krishnan, and D. Ferraiolo, "Translating Natural Language Specifications into Access Control Policies by Leveraging Large Language Models," NIST publication record, Jan. 16, 2025.

[7] Z. Ji et al., “Survey of hallucination in natural language generation,” *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.

[8] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[9] National Institute of Standards and Technology, “Role Based Access Control (RBAC) FAQs.” [Online]. Available: https://csrc.nist.gov/Projects/role-based-access-control/faqs. Accessed: Oct. 22, 2026.

[10] OASIS, “eXtensible Access Control Markup Language (XACML) Version 3.0,” OASIS Standard, Jan. 2013.

[11] A. Gouglidis, A. Kagia, and V. C. Hu, “Model Checking Access Control Policies: A Case Study using Google Cloud IAM,” arXiv:2303.16688, 2023.

[12] National Institute of Standards and Technology, “Security and Privacy Controls for Information Systems and Organizations,” NIST Special Publication 800-53, Rev. 5.

[13] C. Autio et al., “Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile,” NIST AI 600-1, National Institute of Standards and Technology, Jul. 2024. doi: 10.6028/NIST.AI.600-1.

[14] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” arXiv:2302.12173, 2023.